Libtycoin: A Peer-to-Peer Electronic Cash System

Tobin Wyler
tobinwyler@proton.me

Abstract. *A purely peer-to-peer form of electronic cash would let online payments move directly from one party to another without the mediation of any financial institution. Cryptographic signatures provide part of the answer, but the principal benefits collapse the moment a trusted third party is still required to prevent the same coin from being spent twice. We propose a resolution to this double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an unbroken chain of hash-based proof-of-work, producing a record that cannot be altered without recomputing the work. The longest chain serves both as evidence of the sequence of events as it was witnessed, and as proof that the chain emerged from the largest concentration of computational power. So long as a majority of that power remains in the hands of nodes that do not collude against the network, those nodes will extend the longest chain and outrun any attacker. The network itself demands almost no structure. Messages are propagated on a best-effort basis. Nodes leave and rejoin at will, accepting whatever proof-of-work chain is longest as the canonical account of what occurred while they were absent. The design is not new: it is restated here without alteration. What this paper adds is the case for founding such a chain afresh, while participation in it remains open to all.*

1. Introduction

Commerce conducted online has come to rely almost entirely on intermediaries that stand between the two parties to a payment and serve as trusted third parties. Every transaction passes through a single point of authority, capable of recording, restricting or reversing it at will. The arrangement works well enough for most transactions, yet it remains bound by the inherent weaknesses of the trust-based model. Truly irreversible transactions are not possible, since the intermediary cannot avoid arbitrating disputes. This unavoidable mediation raises the cost of every payment, sets a floor on the smallest amount that can practically be exchanged, and forecloses the possibility of casual micropayments altogether. There is a deeper cost still: the loss of any ability to make non-reversible payments for non-reversible services. Where mediation can always be undone, a trivial purchase carries the same demand for trust as a solemn undertaking, and every exchange becomes a matter of faith — faith placed in a party that answers to no one. And because reversal remains available, the demand for trust does not stop at the intermediary; it reaches the counterparties themselves. A seller comes to see in every buyer a payment that might yet be undone, and to ask of him more than the sale itself would ever warrant.

What such an arrangement quietly removes is the possibility of holding value without permission. While several ledgers compete, and while some instrument survives that settles outside all of them, the power to restrict is bounded: whoever is refused may go elsewhere. That bound falls away the moment the keeping of the ledger and the issue of the money become one and the same office, and nothing is left that settles outside the record. Restriction then ceases to be a commercial judgement and becomes an instrument of policy. Whether a sum may be spent at all, upon what, within which borders and before which date, becomes a setting in the ledger rather than a decision of the person holding the money. Nothing in such a design compels it to be used so; nothing in it prevents that either, and the difference rests on assurances alone. As money becomes an entry in someone else's ledger and nothing besides, the freedom to transact comes to rest on the continued goodwill of whoever keeps it — a dependency most people never chose, and rarely even notice.

A certain percentage of fraud is treated as unavoidable. These costs and uncertainties of payment can be reduced through direct, in-person exchange of physical currency, but no equivalent mechanism exists for transactions across a communication channel without a trusted party.

What is needed is an electronic payment system grounded in cryptographic proof rather than in trust, allowing any two willing participants to transact directly with one another, without the necessity of any third party. Transactions that are computationally impractical to reverse would protect sellers from chargeback fraud, and routine escrow arrangements could be implemented to protect buyers when desired.

The design set out here is not new. Satoshi Nakamoto published it in 2008 [9], and the network it describes has run ever since, from the January that followed — which is the strongest thing that can be said of any protocol. Nothing in it wants improving, and we have not attempted to.

What changes over the life of any such chain is not the design but the distance between it and an ordinary person. Issuance is front-loaded by construction: the earlier one arrives, the greater the share a given quantity of work can claim, and that share only ever falls. The securing of the chain concentrates in step, as specialised hardware displaces ordinary machines and the margin narrows to those who operate at scale. Acquisition, as a chain grows valuable, passes increasingly through custodians, exchanges and funds — institutions of the very sort the design was meant to render unnecessary — because value worth stealing becomes troublesome to hold, and the trouble sells readily as a service. None of this is a fault in the design, nor a failure of those who built upon it. It is what success looks like.

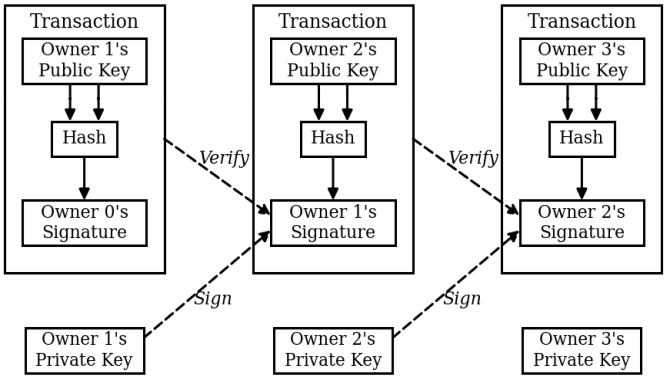
The opening years are therefore the only period in which the terms stand equal for everyone. Coins go to whoever does the work, on whatever machine they happen to own, without permission and without anyone standing in between. That period is not a stage on the way to something better; it is a distinct good, and no chain furnishes it twice. A mature chain does other things well — it holds value, it settles large sums, it endures — and those are the things it should be left to do.

We therefore propose no improvement. We propose to begin again, at the point where participation is still open, with a design repeated deliberately and without alteration. This chain will in its turn stand where the first now stands — not a defect to be engineered around, but the shape of the thing. What matters is that the design remains available, exactly as written, to whoever has need of it. Need, rather than preference, is what warrants taking it up: a beginning is worth having only where it is seldom had. A system that could be founded only once would rest, in the end, on the goodwill of those who happened to be early.

In this paper, we put forward a solution to the double-spending problem using a peer-to-peer distributed timestamp server, which generates computational proof of the chronological order in which transactions occurred. The system is secure so long as honest nodes collectively control more processing power than any cooperating group of attackers.

2. Transactions

An electronic coin, as we define it, is a chain of digital signatures. Each owner passes the coin on by signing, digitally, the hash of the preceding transaction together with the public key of the party who is to receive it, and appending what results to the end of the coin. The signatures may then be checked in sequence, and the chain of ownership with them.

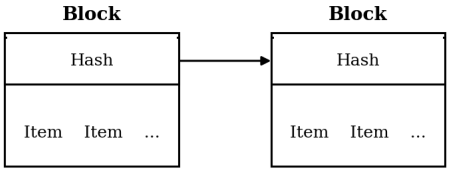


The difficulty, of course, is that the payee cannot independently verify that one of the previous owners has not already signed the same coin over to someone else. The classical remedy introduces a trusted central authority — a mint — that examines every transaction for double-spending. After each payment, the coin must be returned to the mint to be reissued. Only coins issued directly from this mint are trusted not to be double-spent. The flaw is structural: the fate of the entire monetary system rests on the company running the mint, with every single transaction passing through it, exactly as it does today through a bank.

What the payee lacks is a means of knowing that none of those previous owners signed an earlier transaction. It is the first transaction that counts here; whatever is attempted afterwards is disregarded. To establish that a transaction does not exist, one must know of every transaction that does. Under the mint that knowledge sat in a single place, and the mint decided which had arrived first. To reach the same result without a trusted party, every transaction must be announced in public [1], and the participants must have some means of converging upon a single history of the order in which those transactions arrived. The payee needs proof that, at the moment of every transaction, a majority of the network witnessed it as the first one in flight.

3. Timestamp Server

The solution we propose begins with a timestamp server. A timestamp server takes the hash of a block of items to be timestamped and publishes the hash widely, in a manner equivalent to publishing the result in a newspaper or in a public broadcast [2-5] — visible to all, deniable by none. The timestamp proves that the data must have existed at a particular moment, since otherwise it could not have entered the hash. Each timestamp includes the previous timestamp inside its own hash, forming a chain in which every new link reinforces every link that came before it.



This construction has two consequences that matter. First, no entry can be removed without removing every entry that followed it, because each subsequent hash depends on it. Second, no entry can be silently inserted,

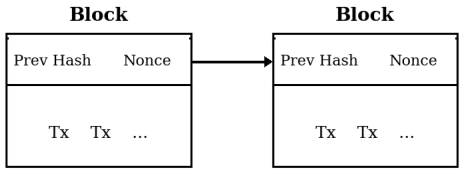
because doing so would shift every hash after the point of insertion, breaking the continuity of the chain in a way that any participant can verify in seconds.

The result is not a record kept by an institution. It is a record that no institution has the power to keep otherwise.

4. Proof-of-Work

A timestamp server run peer-to-peer calls for something other than newspapers and Usenet posts. We use a proof-of-work of the kind set out in Adam Back's Hashcash [6]. The work consists in searching for a value whose hash — SHA-256, say — begins with a stipulated number of zero bits. The effort this demands grows exponentially with the number of zeros stipulated, while the result is confirmed by a single computation.

In our timestamp network the work is done by incrementing a nonce within the block until its hash carries the leading zeros required. The effort once spent, the block cannot be altered without spending it again; and since later blocks are chained behind it, altering it means redoing every block that came after.



Proof-of-work also settles the question of representation in a majority decision. Were the majority counted by one address, one vote, it could be overturned by anyone able to acquire addresses in quantity. Under proof-of-work the ballot is cast by the work itself: a participant weighs in the decision in proportion to the effort he expends, and in no other proportion. Identity confers nothing; nor does the number of machines, nor their kind. The decision of the majority is the longest chain, being the one into which the greatest effort has been sunk. Where honest nodes hold the greater part of that effort, their chain advances faster than any rival. To alter a block already laid down, an attacker must redo its proof-of-work and that of every block since, then draw level with the honest nodes and pass them. Section 11 shows that his odds of managing it fall away exponentially with each block added.

To compensate for variations in hardware speed and the fluctuating interest of nodes joining and leaving the network over time, the proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour. The mechanism self-adjusts. If blocks are produced too quickly, the difficulty rises. If too slowly, it falls. The clock corrects itself.

5. Network

The steps to run the network are as follows:

1) A new transaction is announced to every node that will listen.
2) Each node gathers the transactions it has heard into a candidate block.
3) Each node sets about finding a difficult proof-of-work for that block.
4) Whichever node finds one announces the completed block to the rest.
5) A node adopts the block only once satisfied that every transaction within it is valid and unspent.
6) Adoption is expressed in labour rather than in declaration: nodes begin building the next block upon the hash of the one they have accepted.

A node takes the longest chain to be the correct one and continues to build upon it. Should two nodes publish rival versions of the next block at the same moment, part of the network will hear one first and part the other.

Each works on whichever reached it first, while keeping the rival branch in reserve in case it should outgrow the first. The matter settles itself at the next proof-of-work: one branch draws ahead, and the nodes that had been labouring on the other abandon it for the longer.

A transaction need not reach every node when it is announced. Reaching many is enough; it will find its way into a block soon enough. Block announcements are likewise tolerant of loss. A node that misses one will notice the gap on receiving the next, and ask for what it lacks.

6. Incentive

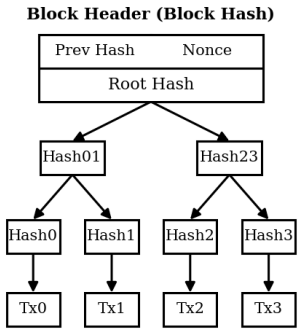
By convention, the opening entry of every block is one of a special kind: it brings a new coin into being, assigned to whoever assembled the block. This serves two purposes at once. It repays the work that secures the network, and it settles the question of issuance in a system that has no issuer — coins enter circulation because work was done, not because any party resolved that they should. The addition of new coins on a schedule fixed in advance, and diminishing as it runs, is the counterpart of miners committing resources to bring gold to market. What is committed here is processor time and electricity.

Fees may fund the same reward. Where the outputs of a transaction are worth less than its inputs, the remainder stands as a fee and is folded into the reward of the block that carries it. Once the predetermined supply has entered circulation in full, the reward may rest on fees alone and issuance ceases altogether. The schedule is settled in advance and known to all; no party stands in a position to revise it by decree.

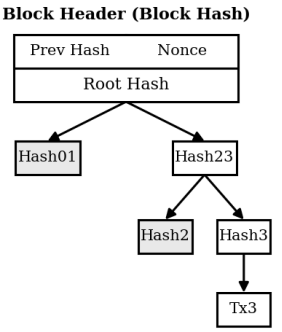
The reward also gives the network reason to expect honesty. An attacker who had gathered more processing power than all the honest nodes together would face a choice: to spend it defrauding people by clawing back payments he has already made, or to spend it producing new coins. The arithmetic favours the second. The rules hand him more new coins than the rest of the network combined, which is worth more than anything he might seize by discrediting the system — and with it the value of everything he already holds.

7. Reclaiming Disk Space

Once a coin’s most recent transaction lies buried beneath a sufficient depth of blocks, the spent transactions preceding it may be dropped and the space recovered. So that this can be done without disturbing the block’s hash, transactions are committed to a Merkle Tree [7][2][5] and only the root is carried in the hash of the block. Branches may then be stubbed away and old blocks compacted. The interior hashes need not be kept.



Transactions Hashed in a Merkle Tree

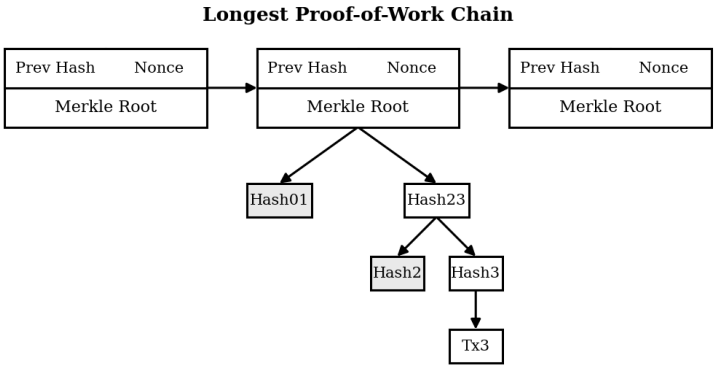


After Pruning Tx0-2 from the Block

A block header carrying no transactions runs to some 80 bytes. At one block every ten minutes, that comes to 80 × 6 × 24 × 365, or 4.2 MB in a year. Set against the storage available on ordinary machines, and against its continued growth, the burden is negligible — even were every header to be held in memory.

8. Simplified Payment Verification

Payments can be verified without running a full node. It is enough to hold the block headers of the longest proof-of-work chain — obtained by querying nodes until one is satisfied that the chain in hand is the longest — together with the Merkle branch tying the transaction to the block that timestamped it. Such a user cannot examine the transaction directly; but in placing it within the chain, they can see that a node accepted it, and every block laid down afterwards confirms that the network did the same.

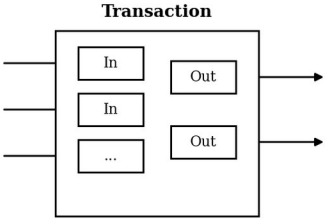


Merkle Branch for Tx3

Verification of this kind holds so long as honest nodes command the network, and grows fragile if an attacker comes to overpower it. A full node checks transactions for itself; the simplified method can be taken in by transactions the attacker has fabricated, for as long as his advantage lasts. One defence is to have the software listen for alerts raised by network nodes when they encounter an invalid block, then fetch the full block along with the flagged transactions and establish the discrepancy for itself. Businesses handling frequent payments will nevertheless prefer nodes of their own, for independence and for speed.

9. Combining and Splitting Value

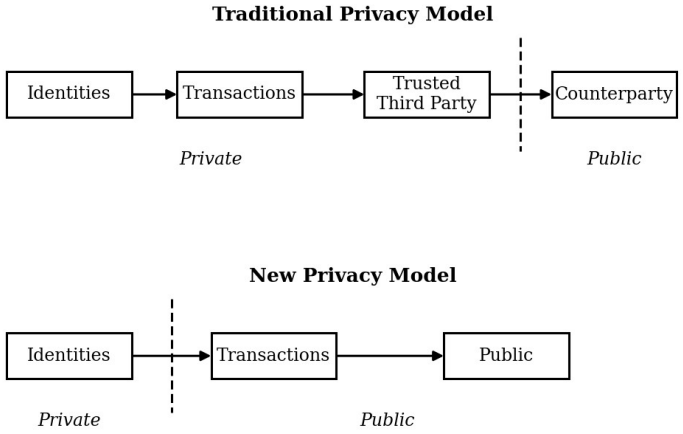
Handling coins one at a time would be possible, but unwieldy: a transfer of any size would demand a separate transaction for every cent. Transactions therefore carry several inputs and several outputs, so that value may be divided and recombined. In the usual case there is one input drawn from a larger earlier transaction, or several inputs gathering smaller sums, and no more than two outputs — one for the payment itself, and one returning the change, if any, to the sender.



Fan-out — a transaction resting upon several others, each of which rests upon many more — poses no difficulty. At no point need anyone extract a complete, self-contained copy of a coin’s history.

10. Privacy

Banking achieves a measure of privacy by restricting who may see what: the parties to a payment, and the institution standing between them. Announcing every transaction in public forecloses that method, but the flow of information may still be interrupted, at a different point — at the identity behind the key rather than at the record of the payment. Anyone may observe that a sum has moved from one key to another; nothing in that record says whose keys they were. This is comparable to the level of disclosure at a stock exchange, where the time and size of individual trades — the tape — are made public, while the identities of the parties behind them are not. A further precaution is to use a fresh key pair for every transaction, so that payments are not gathered under a single visible owner. Some linkage remains unavoidable: a transaction drawing on several inputs must reveal that those inputs shared an owner. The residual risk is that exposing the owner of one key may expose the others alongside it. Where a ledger has a single keeper, no such precaution is available at all: the keeper sees every key, every sum and every party, by construction rather than by breach.



11. Calculations

We turn to an attacker attempting to build an alternate chain faster than the honest one. Success there would not open the system to arbitrary rewriting: he could conjure no value from nothing, nor take coins that were never his. Nodes will not accept an invalid transaction as payment, and honest nodes will never accept a block that carries one. What remains to him is narrower — to revise one of his own transactions and reclaim money he has lately spent.

The contest between the honest chain and the attacker's may be treated as a Binomial Random Walk. A success is the honest chain gaining a block, widening its lead by +1; a failure is the attacker gaining one, closing the gap by −1. The odds of his recovering from a given deficit are those of the Gambler's Ruin [8]. Writing p for the probability that an honest node finds the next block, q for the probability that the attacker does, and q_z for the probability that he ever closes a gap of z blocks:

$$q_z = \begin{cases} 1 & \text{if } p \leq q \\ (q/p)^z & \text{if } p > q \end{cases}$$

Since p > q, the probability falls away exponentially as the number of blocks to be recovered grows. The odds being against him, an attacker who does not lunge forward early finds his chances vanishing as he drops further behind.

We turn next to how long the recipient of a payment must wait before he may be sufficiently certain that the sender can no longer undo it. Suppose the sender to be an attacker who means the recipient to believe

himself paid, and then, after some interval, to return the payment to himself. The recipient will be alerted when it happens; the sender is counting on the alert arriving too late.

The recipient generates a fresh key pair and hands the public key to the sender only shortly before signing. This denies the sender the chance to prepare a chain in advance, working at it until he stands far enough ahead and executing the transaction only then. Once it is sent, the dishonest sender falls to work in secret on a parallel chain holding an altered version of his own transaction.

The recipient waits until the transaction has entered a block and z further blocks have been linked behind it. He cannot know how far the attacker has come; but taking the honest blocks to have arrived at the average expected interval, the attacker's likely progress follows a Poisson distribution of expected value:

$$\lambda = z\,\frac{q}{p}$$

To find the probability that he could still close the gap, we weight the Poisson density at each degree of progress by the probability of recovery from that point:

$$\sum_{k=0}^{\infty} \frac{\lambda^k e^{-\lambda}}{k!} \cdot \begin{cases} (q/p)^{z-k} & \text{if } k \le z \\ 1 & \text{if } k > z \end{cases}$$

Rearranged, so that the infinite tail need not be summed:

$$1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!} \left(1 - (q/p)^{z-k}\right)$$

Converting to C code:

```
#include <math.h>
double AttackerSuccessProbability(double q, int z)
{
    double p = 1.0 - q;
    double lambda = z * (q / p);
    double sum = 1.0;
    int i, k;
    for (k = 0; k <= z; k++)
    {
        double poisson = exp(-lambda);
        for (i = 1; i <= k; i++)
            poisson *= lambda / i;
        sum -= poisson * (1 - pow(q / p, z - k));
    }
    return sum;
}
```

Some computed values show the probability falling away exponentially in z:

```
q=0.1                                   q=0.3
z=0   P=1.0000000                       z=0   P=1.0000000
z=1   P=0.2045873                       z=5   P=0.1773523
z=2   P=0.0509779                       z=10  P=0.0416605
z=3   P=0.0131722                       z=15  P=0.0101008
z=4   P=0.0034552                       z=20  P=0.0024804
z=5   P=0.0009137                       z=25  P=0.0006132
z=6   P=0.0002428                       z=30  P=0.0001522
z=7   P=0.0000647                       z=35  P=0.0000379
z=8   P=0.0000173                       z=40  P=0.0000095
z=9   P=0.0000046                       z=45  P=0.0000024
z=10  P=0.0000012                       z=50  P=0.0000006
```

Solving for P less than 0.1%:

```
P < 0.001
q=0.10   z=5                            q=0.30   z=24
q=0.15   z=8                            q=0.35   z=41
q=0.20   z=11                           q=0.40   z=89
q=0.25   z=15                           q=0.45   z=340
```

12. Conclusion

We have set out a system for electronic transactions that does not rest on trust. The starting point was the familiar one: coins as chains of digital signatures, which settle ownership firmly but leave double-spending unanswered. Our answer is a peer-to-peer network recording a public history of transactions under proof-of-work — a history that becomes computationally impractical to alter, so long as honest nodes hold the majority of processing power. The strength of the network lies in having almost no structure to attack. Nodes act at once and with little coordination. They need no identity, since messages are not routed to any particular destination and need only be delivered on a best-effort basis. They may depart and return as they please, taking the proof-of-work chain as the account of whatever passed in their absence. They vote with their processors: accepting a block by building upon it, rejecting one by declining to. Whatever rules and incentives the system requires can be carried by that same mechanism. The system is, by construction, indifferent to whatever authority might claim the power to issue or withhold money — and to any party, public or private, that might seek to grant or revoke the right to transact.

References

[1] W. Dai, “b-money”, http://www.weidai.com/bmoney.txt, 1998.

[2] H. Massias, X.S. Avila, and J.-J. Quisquater, “Design of a secure timestamping service with minimal trust requirements”, 20th Symposium on Information Theory in the Benelux, May 1999.

[3] S. Haber, W.S. Stornetta, “How to time-stamp a digital document”, Journal of Cryptology, vol 3, no 2, pages 99–111, 1991.

[4] D. Bayer, S. Haber, W.S. Stornetta, “Improving the efficiency and reliability of digital time-stamping”, Sequences II: Methods in Communication, Security and Computer Science, pages 329–334, 1993.

[5] S. Haber, W.S. Stornetta, “Secure names for bit-strings”, Proceedings of the 4th ACM Conference on Computer and Communications Security, pages 28–35, April 1997.

[6] A. Back, “Hashcash — a denial of service counter-measure”, http://www.hashcash.org/papers/hashcash.pdf, 2002.

[7] R.C. Merkle, “Protocols for public key cryptosystems”, Proc. 1980 Symposium on Security and Privacy, IEEE Computer Society, pages 122–133, April 1980.

[8] W. Feller, “An introduction to probability theory and its applications”, 1957.

[9] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”, 2008.